

Smart Blind Assistant

Rapport Final de Projet

Système Intelligent d'Assistance aux Personnes Malvoyantes

Backend FastAPI · Computer Vision · NLP · Application Flutter

Mai 2026

Réalisé par

BADO Yipène Ange Cenacle

ISSOUG El Mehdi

Projet Computer Vision & NLP · S6 · Mai 2026

Table des matières

Résumé Exécutif	3
1 Présentation du Projet	3
1.1 Contexte et Problématique	3
1.2 Objectifs du Projet	4
1.3 Utilisateurs Cibles	4
1.4 Fonctionnalités du Système	4
1.5 Avantages et Bénéfices	5
2 Architecture Globale de la Solution	5
2.1 Vue d'ensemble	5
2.2 Principe architectural fondamental	6
3 Backend FastAPI — Moteur Intelligent	7
3.1 Pourquoi FastAPI?	7
3.2 Le Pipeline — Route Centrale	7
3.3 API REST — Endpoints	8
3.4 Gestion des Images Annotées	9
4 Module Computer Vision	9
4.1 Architecture des Modèles	9
4.2 YOLO — You Only Look Once	9
4.3 Fine-tuning pour la Détection de Dangers	10
4.4 Depth Anything V2 — Estimation de Distance	10
4.5 detector.py — Surveillance d'Obstacles	11
4.5.1 Système de priorité des alertes	11
4.6 finder.py — Recherche et Guidage d'Objet	11
4.6.1 Système de 5 zones de direction	12
4.7 localizer.py — Identification de Pièce	12
5 Module NLP — Compréhension du Langage	12
5.1 Défi de la Compréhension Vocale	13
5.2 Architecture Combinée à Trois Niveaux	13
5.3 SentenceTransformer — Compréhension Sémantique	13
5.4 spaCy — Analyse Grammaticale	13
5.5 Extraction d'Entité — Approche Sémantique Sans Liste	14
5.6 Champ « source » — Traçabilité	15
6 Module Navigation	15
6.1 Principe	15
6.2 Configuration de la Maison	15
6.3 Algorithme de Recherche de Chemin — BFS	16
6.4 Instructions Dynamiques selon l'Orientat	16

7	Module Safety — Alertes SOS	16
7.1	Déclenchement	16
7.2	Actions du Module	16
7.3	Réponse Flutter	16
8	Interface Aidant — caregiver.py (Streamlit)	17
8.1	Pourquoi une Interface Séparée ?	17
8.2	Fonctionnalités	17
8.3	Règle d’Unicité par Email	18
9	Application Mobile Flutter	18
9.1	Philosophie	18
9.2	Architecture	18
9.3	Surveillance d’Obstacles — Boucle Permanente	18
9.4	Commande Vocale — Flux STT	19
9.5	Annulation des Actions	19
9.6	Modes de l’Application	19
9.7	Dépendances Flutter	19
10	Performance et Optimisation	20
10.1	Chargement des Modèles — Pattern Singleton	20
10.2	Exécution Parallèle des Modèles YOLO	21
10.3	Gestion des Fichiers Temporaires	21
10.4	Envoi d’Images par Bytes	21
11	Gestion des Erreurs	21
11.1	Côté Backend	21
11.2	Côté Flutter	21
12	Limitations et Améliorations Futures	22
12.1	Limitations Actuelles	22
12.2	Améliorations Futures	22
	Conclusion	23

Résumé Exécutif

Smart Blind Assistant est un système complet d'assistance aux personnes malvoyantes, combinant intelligence artificielle, vision par ordinateur et traitement du langage naturel. Le système se compose de trois parties interdépendantes : un backend Python FastAPI constituant le moteur intelligent, une application mobile Flutter servant d'interface vocale pour l'utilisateur final, et une interface Streamlit dédiée à la configuration par un aidant.

Le système permet à un utilisateur malvoyant d'interagir naturellement avec son environnement via la voix : retrouver des objets perdus, identifier sa position dans la maison, naviguer de pièce en pièce et déclencher des alertes d'urgence. La détection d'obstacles fonctionne en permanence en arrière-plan pour prévenir les dangers.

Composant	Technologie	Rôle	Statut
Backend API	Python / FastAPI	Intelligence artificielle	Terminé
Computer Vision	YOLOv8 + Depth Anything V2	Détection et guidage	Terminé
NLP	SentenceTransformer + spaCy	Compréhension vocale	Terminé
Navigation	BFS + Graphe maison	Guidage pièce à pièce	Terminé
App Mobile	Flutter / Dart	Interface utilisateur	Terminé
Interface Aidant	Streamlit (Python)	Configuration maison	Terminé

1 Présentation du Projet

1.1 Contexte et Problématique

Au Maroc, plusieurs personnes vivent avec une déficience visuelle. Parmi elles, des centaines de milliers sont en situation de cécité totale ou partielle, rendant les activités quotidiennes complexes et parfois dangereuses. Se déplacer dans sa maison, retrouver des objets du quotidien, identifier une pièce inconnue ou appeler à l'aide en cas d'urgence sont des défis permanents.

Les solutions traditionnelles — canne blanche, chien guide — sont précieuses mais limitées : elles ne peuvent pas identifier un objet spécifique, lire l'environnement visuel ou guider vers une destination précise dans un espace intérieur. Les assistants vocaux existants (Alexa, Google Home) ne sont pas conçus pour la navigation spatiale et la détection d'obstacles

en temps réel.

La problématique centrale

Comment permettre à une personne malvoyante d'interagir de manière autonome et sécurisée avec son environnement intérieur, en utilisant uniquement sa voix et son smartphone, sans nécessiter de compétences techniques particulières ?

1.2 Objectifs du Projet

- Détecter et signaler automatiquement les obstacles dans l'environnement immédiat de l'utilisateur.
- Permettre la recherche vocale d'objets du quotidien et guider l'utilisateur jusqu'à l'objet trouvé.
- Identifier la pièce courante par analyse visuelle de l'environnement.
- Guider l'utilisateur de pièce en pièce grâce à un plan de maison personnalisé.
- Déclencher des alertes d'urgence rapidement et efficacement.
- Offrir une interface de configuration simple à destination des aidants.

1.3 Utilisateurs Cibles

Profil	Description et besoins
Personne malvoyante	Utilisateur principal — interagit exclusivement par la voix, n'utilise pas l'écran
Aidant / proche	Configure la maison via l'interface Streamlit (une seule fois)
Personnel médical	Peut être contacté via le module SOS en cas d'urgence
Jury académique / client	Évalue la pertinence technique et fonctionnelle de la solution

1.4 Fonctionnalités du Système

Fonctionnalité	Description détaillée
Surveillance d'obstacles	Analyse continue de la caméra toutes les 5s — alertes vocales avec position et distance
Détection de dangers	Feu, couteau, escaliers — alerte prioritaire quelle que soit la distance

Fonctionnalité	Description détaillée
Recherche d'objet	Guidage pas à pas vers un objet spécifique via instructions vocales (5 zones de direction)
Localisation de pièce	Identification de la pièce courante par analyse des objets visibles
Navigation guidée	Calcul du chemin optimal entre deux pièces et instructions vocales dynamiques
Alerte SOS	Déclenchement vocal, log horodaté, simulation d'appel aux secours, vibration forte
Compréhension vocale	Reconnaissance des intentions en français naturel — pas de commandes à mémoriser
Configuration maison	Interface aidant pour définir les pièces et connexions — sauvegardé par email

1.5 Avantages et Bénéfices

- Interaction 100% vocale : aucun besoin de voir ou toucher l'écran.
- Compréhension du langage naturel : « je ne retrouve pas mon lit » fonctionne aussi bien que « cherche le lit ».
- Adaptabilité : chaque maison est configurable, pas de plan prédéfini.
- Sécurité active : détection des dangers (feu, couteau) en permanence, pas seulement sur demande.
- Architecture modulaire : chaque fonctionnalité est indépendante et évolutive.
- Aucune dépendance matérielle spécifique : fonctionne sur tout smartphone Android récent.

2 Architecture Globale de la Solution

2.1 Vue d'ensemble

La solution est structurée en trois couches distinctes qui communiquent via une API REST. Cette séparation stricte des responsabilités garantit la maintenabilité, l'évolutivité et la clarté du code.

Couche	Composant	Rôle
Présentation	App Flutter (Android)	Interface vocale utilisateur — STT, TTS, caméra
Présentation	Streamlit (caregiver.py)	Interface aidant — configuration de la maison
Traitement	Backend FastAPI (main.py)	Orchestration — routage vers les modules IA
Intelligence	Computer Vision (cv/)	Détection, recherche, localisation par image
Intelligence	NLP (nlp/)	Compréhension et analyse des commandes vocales
Intelligence	Navigation (navigation/)	Calcul de chemins et instructions de guidage
Urgence	Safety (safety/)	Gestion des alertes SOS
Données	navigation/houses/	Configurations de maisons par utilisateur

2.2 Principe architectural fondamental

L'architecture repose sur un principe central : toute l'intelligence réside dans le backend. L'application Flutter est un client passif qui exécute les instructions reçues sans les interpréter.

- Flutter ne contient aucune logique métier — il envoie des données, reçoit des instructions, les exécute.
- Le backend décide de tout : quel module appeler, quand boucler, quand vibrer, quand afficher l'urgence.
- Les mises à jour du système ne nécessitent pas de mise à jour de l'application mobile.

Exemple concret

Quand l'utilisateur dit « je cherche ma télécommande », Flutter envoie la phrase et une image au backend. Le backend retourne : `tts_message='Tournez à droite', action.loop=true, action.loop_interval=5`. Flutter lit le message et programme un rappel toutes les 5 secondes — sans savoir pourquoi.

3 Backend FastAPI — Moteur Intelligent

3.1 Pourquoi FastAPI ?

FastAPI est un framework Python moderne conçu pour les API hautes performances. Il a été retenu pour plusieurs raisons techniques :

- Performances : comparable à Node.js et Go, bien supérieur à Flask ou Django REST pour les opérations asynchrones.
- Gestion native des fichiers multipart : essentiel pour recevoir les images envoyées par Flutter.
- Documentation automatique : une interface Swagger est générée automatiquement sur `/docs`.
- Typage fort avec Pydantic : validation automatique des données entrantes.
- BackgroundTasks : permet de supprimer les fichiers temporaires en arrière-plan sans bloquer la réponse.

3.2 Le Pipeline — Route Centrale

La route `/pipeline` est le cœur du système. Elle orchestre l'ensemble du traitement en une seule requête : NLP pour comprendre la commande, puis routage vers le bon module CV ou navigation selon l'intention détectée.

Étape	Module	Description
1	Prétraitement	Sauvegarde de l'image, nettoyage de la phrase (lowercase + strip)
2	NLP / intent.py	Détection de l'intention (FIND_OBJECT, NAVIGATE, LOCATE_ROOM, SOS, UNKNOWN)
3	Extraction entité	Extraction de l'objet ou du lieu mentionné dans la phrase
4	Routage	Appel du module CV ou navigation selon l'intention
5	Annotation	Dessin des résultats sur l'image (bounding boxes, labels)
6	Réponse	Construction de la réponse avec <code>tts_message</code> + <code>action</code> + <code>image_url</code>

Étape	Module	Description
7	Nettoyage	Suppression des fichiers temporaires en arrière-plan (30s)

La réponse contient systématiquement un objet **action** qui dit à Flutter exactement quoi faire : boucler, vibrer, stopper la surveillance, afficher l'urgence. Flutter n'a aucune décision à prendre.

3.3 API REST — Endpoints

Méthode	Route	Description
GET	/	Vérification que le serveur est opérationnel
POST	/pipeline	Route principale : phrase + image → NLP + CV + instructions Flutter
POST	/detect-intent	NLP seul : analyse une phrase et retourne l'intention + entité
POST	/find-object	Recherche un objet dans une image et génère des instructions de guidage
POST	/where-am-i	Identifie la pièce courante depuis une image caméra
POST	/detect-obstacle	Détecte les obstacles et génère les alertes vocales prioritaires
POST	/navigate	Navigation guidée : détecte pièce actuelle + génère chemin + instructions
POST	/sos	Déclenche une alerte d'urgence avec log et simulation d'appel
POST	/house/save	Sauvegarde la configuration maison d'un utilisateur (une fois par email)
GET	/house	Récupère la configuration maison d'un utilisateur

Méthode	Route	Description
PUT	/house/update	Met à jour une configuration maison existante

3.4 Gestion des Images Annotées

En mode développement et test, chaque réponse des routes CV contient une `image_url` : l'image de la caméra avec les bounding boxes et labels dessinés, accessible via une URL temporaire. Ces images sont sauvegardées dans `static/` et supprimées automatiquement après 30 secondes via le mécanisme BackgroundTasks de FastAPI.

- Les noms de fichiers sont des UUIDs générés aléatoirement — pas de conflits entre requêtes simultanées.
- En production avec Flutter, ce champ peut être ignoré — il ne sert qu'au debug visuel.

4 Module Computer Vision

4.1 Architecture des Modèles

Le module CV utilise deux types de modèles d'intelligence artificielle complémentaires :

Modèle	Type	Rôle
YOLOv8n (COCO)	Détection d'objets — 80 classes	Usage général : meubles, personnes, appareils
YOLOv8n Custom (best.pt)	Détection d'objets — 4 classes	Dangers : fire, hunman-fire, knife, stairs
Depth Anything V2 Small	Estimation de profondeur monoculaire	Distance des obstacles sans capteur dédié

4.2 YOLO — You Only Look Once

YOLO est un algorithme de détection d'objets en temps réel. Contrairement aux approches classiques qui analysent l'image en plusieurs passes, YOLO effectue une seule passe sur l'image entière et retourne simultanément la localisation (bounding box) et la classe de chaque objet détecté.

Pourquoi YOLOv8n ?

La version « nano » (n) de YOLOv8 est la plus légère du framework — idéale pour un déploiement sur serveur sans GPU dédié. Elle offre un bon compromis entre vitesse (~30ms par image sur CPU) et précision suffisante pour les applications d'assistance.

4.3 Fine-tuning pour la Détection de Dangers

YOLOv8 standard ne détecte pas les éléments critiques pour la sécurité d'un malvoyant. Un modèle custom a été entraîné :

- Création d'un dataset sur Roboflow : fusion de 3 datasets spécialisés (escaliers, feu, couteau) — 5 641 images, 4 classes.
- Entraînement sur Google Colab (GPU Tesla T4) : 50 epochs, YOLOv8n, paramètre freeze=10.
- Freeze=10 : les 10 premières couches du réseau sont gelées, préservant la connaissance générale tout en adaptant les couches supérieures aux nouvelles classes.

Catastrophic Forgetting — Problème et Solution

Le fine-tuning initial a effacé les 80 classes COCO originales (phénomène de « catastrophic forgetting »). Solution retenue : deux modèles en parallèle — `yolov8n.pt` pour les 80 classes COCO, `best.pt` pour les 4 classes de danger. Exécution simultanée via `ThreadPoolExecutor` (gain : 50–80ms vs 200ms en séquentiel).

4.4 Depth Anything V2 — Estimation de Distance

Connaître la distance d'un obstacle est crucial pour évaluer le danger. Sans capteur de profondeur dédié (comme LiDAR), le système utilise Depth Anything V2, un modèle de profondeur monoculaire qui estime la distance depuis une seule image.

Approche	Description	Verdict
Table de référence (TAILLE_REF)	Taille connue par classe × pixels = distance estimée	Abandonnée — liste fermée
Hauteur bbox relative	Plus le bbox est grand = plus l'objet est proche	Fallback — universel mais imprécis
Modèle sténopé (pinhole)	Focale × taille réelle / pixels = distance métrique	Abandonnée — tailles requises
Depth Anything V2 Small	Carte de profondeur complète par IA — ~50ms	Retenue — universelle

Approche	Description	Verdict
ARCore / ARKit	Profondeur native du smartphone	Option future

Un filtre GaussianBlur est appliqué sur la carte de profondeur avant lecture pour stabiliser les valeurs et éviter les variations dues aux textures ou couleurs des objets.

4.5 detector.py — Surveillance d'Obstacles

Ce module analyse chaque frame envoyée par Flutter (toutes les 5 secondes) et retourne une liste d'obstacles avec leurs alertes vocales.

4.5.1 Système de priorité des alertes

Priorité	Classes concernées	Message vocal généré
4	fire	« Danger extrême ! Feu détecté, reculez immédiatement ! »
3	knife	« Danger ! Couteau détecté, soyez prudent ! »
2	stairs	« Attention ! Escaliers, arrêtez-vous ! »
1	tout obstacle proche	« Obstacle détecté, [objet_fr] à votre [direction]. »
0	objet loin	Aucune alerte générée

Suppression des doublons (NMS inter-modèles)

Les deux modèles YOLO peuvent détecter le même objet simultanément. Un algorithme de NMS (Non-Maximum Suppression) basé sur l'IoU (Intersection over Union, seuil 0.5) fusionne les détections redondantes en conservant uniquement celle avec la meilleure confiance.

4.6 finder.py — Recherche et Guidage d'Objet

Ce module guide l'utilisateur pas à pas vers un objet spécifique. Il est appelé en boucle par Flutter toutes les 5 secondes jusqu'à ce que l'objet soit atteint.

4.6.1 Système de 5 zones de direction

Zone (ratio horizontal)	Instruction vocale générée
< 20% — extrême gauche	« Tournez franchement à gauche vers [objet]. »
20–40% — légèrement gauche	« Tournez légèrement à gauche, [objet] est presque là. »
40–60% — centre	« [objet] est droit devant vous, avancez. » / « Vous y êtes, tendez la main. »
60–80% — légèrement droite	« Tournez légèrement à droite et avancez vers [objet]. »
> 80% — extrême droite	« Tournez franchement à droite vers [objet]. »

La condition d'arrivée est : distance=« très proche » ET position=« centre ». Le champ `reached=true` est alors retourné — Flutter arrête la boucle de rappel.

4.7 `localizer.py` — Identification de Pièce

Ce module identifie la pièce courante en analysant les objets visibles dans l'image. Chaque pièce possède une « signature » d'objets caractéristiques avec des poids de confiance.

Niveau de confiance	Message TTS généré	Exemple
$\geq 70\%$	« Vous êtes dans la [pièce]. »	Lit, télé, horloge → chambre 85%
40–69%	« Je pense que vous êtes dans la [pièce]. »	Ordinateur → bureau 55%
< 40%	« Vous êtes peut-être dans la [pièce], je ne suis pas certain. »	Chaise seule → 20%

5 Module NLP — Compréhension du Langage

5.1 Défi de la Compréhension Vocale

Le module NLP doit comprendre des phrases françaises naturelles et variées, extraire l'intention de l'utilisateur et identifier l'objet ou le lieu mentionné — le tout sans forcer l'utilisateur à mémoriser des commandes spécifiques.

5.2 Architecture Combinée à Trois Niveaux

Niveau	Système	Rôle
1	SentenceTransformer (MiniLM)	Similarité sémantique avec les exemples — très efficace pour les formulations connues
2	spaCy fr_core_news_sm	Analyse grammaticale — détecte le verbe d'intention indépendamment de l'objet
3	Fallback entité	Si ni ST ni spaCy : détecte un nom d'objet ou de lieu dans la phrase

Exemple illustratif

« Je ne retrouve pas mon lit » → ST score 0.64 (< 0.72 , insuffisant) → spaCy détecte verbe « retrouver » → FIND_OBJECT. « Je cherche la tv » → ST score 0.58 → spaCy détecte « chercher » → FIND_OBJECT → extraction entité « tv » par lemmatisation.

5.3 SentenceTransformer — Compréhension Sémantique

Le modèle `paraphrase-multilingual-MiniLM-L12-v2` transforme une phrase en vecteur mathématique représentant son sens. La similarité cosinus entre ce vecteur et les exemples de chaque intention détermine l'intention la plus probable.

- Seuil de score : 0.72 — en dessous, l'intention n'est pas assez certaine.
- Seuil de marge : 0.05 — l'écart entre la 1ère et la 2ème intention doit être suffisant.
- Correction du bug « bonjour → NAVIGATE » : le seuil initial à 0.5 classait « bonjour » comme NAVIGATE avec un score de 0.67. Relevé à 0.72, il retourne correctement UNKNOWN.

5.4 spaCy — Analyse Grammaticale

Verbes détectés	Intention déduite	Exemple
trouver, chercher, retrouver, perdre...	FIND_OBJECT	« je ne retrouve pas mon lit »
aller, guider, emmener, conduire...	NAVIGATE	« guide-moi vers la cuisine »
situer, être (+ où)...	LOCATE_ROOM	« je suis où là »

5.5 Extraction d'Entité — Approche Sémantique Sans Liste

Étape	Système	Action
1	spaCy (lemmatisation)	Extrait le nom COD de la phrase (« je cherche MA TV » → « tv »)
2	SentenceTransformer	Calcule la similarité entre « tv » et les 84 classes YOLO
3	Matching	Retourne la classe YOLO la plus proche si score ≥ 0.45

Phrase utilisateur	Entité extraite	Classe YOLO trouvée
« je cherche la tv »	tv	tv (score ~ 0.95)
« je cherche mon frigo »	frigo	refrigerator (score ~ 0.72)
« je cherche mon chien »	chien	dog (score ~ 0.85)
« guide-moi vers la cuisine »	cuisine	pièce (pas de classe YOLO)
« je cherche mon robot »	robot	None (score trop faible)

Avantage multilingue

Le modèle multilingue comprend que « frigo » en français correspond à « refrigerator » en anglais (les classes YOLO sont en anglais). Aucun dictionnaire de traduction n'est nécessaire — la similarité sémantique cross-linguale gère automatiquement cette correspondance.

5.6 Champ « source » — Traçabilité

Valeur	Signification
sentence_transformer	SentenceTransformer confiant — résultat fiable
spacy_verbe	ST insuffisant, spaCy a pris le relais via l'analyse du verbe
fallback_entite	Ni ST ni spaCy — un objet ou lieu a été trouvé dans la phrase
unknown	Aucun niveau n'a pu déterminer l'intention

6 Module Navigation

6.1 Principe

Le module navigation guide l'utilisateur de sa position actuelle vers une pièce cible. Contrairement à la navigation GPS qui nécessite un signal et une cartographie externe, ce système utilise un graphe de connexions défini par l'aidant et la caméra pour détecter la position actuelle.

6.2 Configuration de la Maison

La maison est modélisée comme un graphe orienté où les nœuds sont les pièces et les arêtes sont les connexions avec leurs directions.

- Chaque connexion contient : pièce source, pièce destination, angle absolu (0° =Nord, 90° =Est, 180° =Sud, 270° =Ouest).
- Les angles absolus permettent de calculer dynamiquement gauche/droite/avancez selon l'orientation réelle de l'utilisateur (boussole smartphone).
- Un fichier `house_default.json` sert de configuration de démonstration avec 5 pièces.

6.3 Algorithme de Recherche de Chemin — BFS

Pourquoi BFS et non Dijkstra ?

Dijkstra est optimisé pour les graphes pondérés où les arêtes ont des distances différentes. Dans une maison, toutes les connexions ont le même « coût » — passer d'une pièce à une autre demande le même effort. BFS donne le même résultat optimal avec une implémentation plus simple et plus efficace.

La position de départ est détectée automatiquement par le localizer depuis la caméra. L'utilisateur dit simplement « emmène-moi à la cuisine » sans préciser où il se trouve.

6.4 Instructions Dynamiques selon l'Orientation

Heading utilisateur	Angle cible maison	Instruction générée
90° (regarde Est)	0° (Nord)	« Tournez à gauche. »
90° (regarde Est)	90° (Est)	« Avancez. »
90° (regarde Est)	180° (Sud)	« Tournez à droite. »
0° (regarde Nord)	90° (Est)	« Tournez à droite. »

7 Module Safety — Alertes SOS

7.1 Déclenchement

Le module SOS se déclenche dès qu'une phrase d'urgence est détectée par le NLP : « au secours », « j'ai besoin d'aide », « appelle le 15 », « je suis tombé », « urgence », « help »...

7.2 Actions du Module

- Log horodaté complet : timestamp ISO, identifiant utilisateur, phrase prononcée, coordonnées GPS (si disponibles).
- Simulation d'appel d'urgence : log console « Appel vers 15/18/112 en cours » — remplaçable par un vrai appel via Twilio en production.
- Notification contacts : simulation de notification aux proches.
- Retour Flutter complet : `vibration='strong', son_alarme=true, afficher_ecran_urgence=true` message retour vocal.

7.3 Réponse Flutter

La réponse SOS déclenche dans Flutter : vibration forte (1000ms), navigation vers SosScreen (fond rouge, icône urgence), message vocal « SOS déclenché, les secours ont été alertés, restez en ligne. »

8 Interface Aidant — caregiver.py (Streamlit)

8.1 Pourquoi une Interface Séparée ?

La configuration de la maison est une tâche complexe qui nécessite de voir l'écran et de réfléchir au plan de la maison. Cette tâche est faite par un aidant (proche, personnel médical) une seule fois, pas par l'utilisateur malvoyant. Une interface web Streamlit séparée est donc plus adaptée qu'une fonctionnalité dans l'app mobile.

8.2 Fonctionnalités

Fonctionnalité	Description
Identification par email	Email unique de l'utilisateur — validé par regex, converti en nom de fichier
Ajout de pièces	Ajout une par une avec bouton, suppression individuelle
Définition des connexions	De / Vers / Direction (Nord/Est/Sud/Ouest) avec icônes directionnelles
Aide contextuelle	Expander avec schéma ASCII expliquant les points cardinaux
Aperçu JSON	Visualisation en temps réel de la configuration générée
Sauvegarde	POST /house/save — refusé si email déjà utilisé
Modification	PUT /house/update — pour corriger une configuration existante
Consultation	GET /house — charger et afficher la config d'un utilisateur
Copie de configuration	Copier la maison d'un utilisateur vers un autre email
Réinitialisation	Vider le formulaire pour recommencer

8.3 Règle d'Unicité par Email

Un email ne peut être utilisé qu'une seule fois pour `/house/save`. Cette règle évite les écrasements accidentels. Pour modifier une configuration existante, l'aidant doit utiliser le bouton « Modifier » qui appelle `/house/update`.

Conversion email → fichier

L'email `farou@gmail.com` est converti en `house_farou_gmail_com.json` via `re.sub(r'[^\\w]', '_', email.lower())`. Cette conversion est identique dans Streamlit et dans le backend — les deux pointent vers le même fichier.

9 Application Mobile Flutter

9.1 Philosophie

L'application Flutter est un client vocal passif. Elle ne contient aucune logique métier — elle se contente d'exécuter les instructions reçues du backend. Cette philosophie garantit une application légère, stable et facile à maintenir.

9.2 Architecture

Fichier	Rôle
<code>lib/main.dart</code>	Initialisation — caméra + lancement de <code>HomeScreen</code>
<code>lib/config/api_config.dart</code>	URL backend — <code>10.0.2.2 :8000</code> (émulateur) ou IP locale (vrai téléphone)
<code>lib/services/api_service.dart</code>	Couche réseau — <code>pipeline()</code> , <code>detectObstacle()</code> , <code>cancel()</code> avec timeout 15s
<code>lib/screens/home_screen.dart</code>	Écran principal — orchestration caméra, STT, TTS, timers
<code>lib/screens/sos_screen.dart</code>	Écran urgence — fond rouge, message vocal, bouton annuler

9.3 Surveillance d'Obstacles — Boucle Permanente

Un `Timer.periodic` se déclenche toutes les 5 secondes. Une frame est capturée et envoyée à `/detect-obstacle`. La réponse contient un champ `priorite` (0-5) et `interrupt` (bool) :

- Si `interrupt=true` ou `priorite` ≥ 4 : danger extrême — le TTS en cours est interrompu immédiatement.
- Si `mode=normal` et TTS libre : l'alerte est lue normalement.
- Si `mode=find_object` ou `navigate` : les alertes normales sont ignorées (le guidage a la priorité).

9.4 Commande Vocale — Flux STT

L'utilisateur appuie sur le bouton micro. Le flux STT suit ces étapes :

- Vérification que le STT est initialisé et disponible.
- Arrêt du TTS en cours + délai 100ms pour éviter l'auto-capture du message vocal.
- Message « Je vous écoute » + attente fin du TTS avant de démarrer l'écoute.
- Écoute avec `pauseFor=3s` (silence = fin) et timeout global 12s.
- Détection des mots d'annulation avant d'appeler le pipeline.
- Appel `/pipeline` avec la phrase et une frame caméra.

9.5 Annulation des Actions

L'utilisateur peut annuler une action en cours (recherche, navigation) de deux façons :

- Appui long sur le bouton micro — déclenche `_cancelCurrentAction()` directement.
- Commande vocale — mots-clés : stop, annuler, arrête, stopper, quitter, terminer.

9.6 Modes de l'Application

Mode	Couleur	Icône	Comportement
normal	Blanc	mic_none	Surveillance active, prêt à écouter
find_object	Bleu	stop	Recherche en cours, rappel /pipeline toutes les 5s
navigate	Bleu	stop	Navigation en cours, rappel /pipeline toutes les 5s
sos	Rouge	stop	Urgence déclenchée, SosScreen affiché

9.7 Dépendances Flutter

Package	Version	Rôle
camera	^0.11.0	Capture de frames en temps réel — bytes directs en mémoire
speech_to_text	^7.0.0	STT natif — reconnaissance vocale offline/online
flutter_tts	^4.0.0	TTS natif — voix française automatiquement sélectionnée
http	^1.2.0	Requêtes HTTP multipart — bytes + champs texte
vibration	^2.0.0	Vibration haptique 1000ms pour SOS
permission_handler	^11.0.0	Caméra, microphone, internet
flutter_compass	^0.8.0	Boussole pour heading navigation

10 Performance et Optimisation

10.1 Chargement des Modèles — Pattern Singleton

Tous les modèles IA sont chargés une seule fois au démarrage et maintenus en mémoire via le pattern Singleton Thread-Safe. Un `threading.Lock()` avec double vérification évite les chargements parallèles si plusieurs requêtes arrivent simultanément au démarrage.

Modèle	Temps de chargement	Mémoire approximative
SentenceTransformer (MiniLM)	~3s (cache local)	~50 MB
spaCy fr_core_news_sm	~1s	~15 MB
YOLOv8n COCO	~2s	~12 MB
YOLOv8n Custom (best.pt)	~2s	~12 MB
Depth Anything V2 Small	~5s (cache HuggingFace)	~100 MB

Modèle	Temps de chargement	Mémoire approximative
Embeddings YOLO (84 classes)	~2s (au démarrage)	Négligeable

10.2 Exécution Parallèle des Modèles YOLO

Les deux modèles YOLO sont exécutés simultanément via `ThreadPoolExecutor(max_workers=2)`. Sur GPU RTX 3050 : ~50–80ms en parallèle contre ~200ms en séquentiel — gain de 60% environ.

10.3 Gestion des Fichiers Temporaires

Chaque image reçue est sauvegardée avec un UUID comme nom de fichier (pas de conflits entre requêtes concurrentes). Les images d'entrée sont supprimées immédiatement après traitement. Les images annotées sont supprimées après 30 secondes via `BackgroundTasks`.

10.4 Envoi d'Images par Bytes

Flutter envoie les frames en bytes directement depuis la mémoire (`Uint8List`) via `MultipartFile.fromBytes()`, évitant les écritures sur le disque du téléphone. Cela réduit la latence et la consommation batterie.

11 Gestion des Erreurs

11.1 Côté Backend

- Toutes les routes CV et navigation sont encapsulées dans des blocs `try/except` — une erreur interne retourne un JSON structuré avec `status='error'` et message d'explication.
- Si la pièce actuelle est « inconnue » dans `/navigate`, une réponse claire est retournée avec `tts_message` approprié.
- Si la maison n'existe pas pour un `device_id`, le backend charge automatiquement `house_default.json`.

11.2 Côté Flutter

- Timeout 15s sur tous les appels HTTP — en cas de dépassement, « Délai dépassé, réessayez. »
- Erreur de connexion réseau — « Erreur de connexion au serveur. »
- Caméra non disponible — message vocal au démarrage + dégradation gracieuse.

- STT non disponible — « Microphone non disponible » + bouton désactivé.
- Vérification `if(mounted)` avant chaque `setState` — évite les erreurs de lifecycle.

12 Limitations et Améliorations Futures

12.1 Limitations Actuelles

Limitation	Impact	Gravité
YOLO ne détecte pas clés, lunettes, portefeuille	finder retourne « non trouvé »	Important
Depth Anything V2 imprécis pour 2 objets proches	Instructions légèrement approximatives	Modéré
STT nécessite Google API (connexion internet)	Pas de reconnaissance offline	Important
spaCy fr_core_news_sm — modèle léger	Lemmatisation parfois imprécise	Faible
SOS simulé (pas d'appel réel)	Démo uniquement	Important en production
Heading boussole non intégré au pipeline	Navigation sans orientation réelle	Important

12.2 Améliorations Futures

- Intégration ARCore/ARKit : profondeur native centimétrique du smartphone.
- Modèle NLP amélioré : `paraphrase-multilingual-mpnet-base-v2` pour une meilleure sémantique.
- STT offline : intégration de Vosk ou Whisper pour fonctionnement sans internet.
- Route `/cancel` backend : implémentation complète pour gérer l'annulation côté serveur.
- SOS réel : intégration Twilio pour appels téléphoniques d'urgence.
- Boussole Flutter : intégration `flutter_compass` dans les appels `/pipeline`.
- Interface aidant dans Flutter : écran de configuration intégré à l'app mobile.
- Modèle YOLO étendu : détection des objets personnels (clés, lunettes, portefeuille).

Conclusion

Smart Blind Assistant est une solution complète et fonctionnelle qui répond à la problématique initiale : permettre à une personne malvoyante d’interagir de manière autonome et sécurisée avec son environnement intérieur via son smartphone.

Le système combine trois domaines complémentaires de l’intelligence artificielle — vision par ordinateur, traitement du langage naturel et navigation par graphe — dans une architecture modulaire, extensible et maintenable. La séparation stricte entre le backend intelligent et le client Flutter passif garantit une évolutivité maximale.

Les choix technologiques ont été guidés par trois principes : robustesse (architecture combinée ST+spaCy, deux modèles YOLO, fallbacks multiples), universalité (Depth Anything V2, extraction d’entité sémantique sans liste) et accessibilité (interaction 100% vocale, configuration déléguée à l’aidant).

Le système est opérationnel et testé sur les cas d’usage principaux. Les limitations identifiées (détection d’objets personnels, STT offline, navigation avec boussole) constituent les axes d’amélioration naturels pour les phases suivantes du projet.

Smart Blind Assistant — Rapport Final — Projet S6 — Mai 2026
BADO Yipène Ange Cenacle · ISSOUG El Mehdi